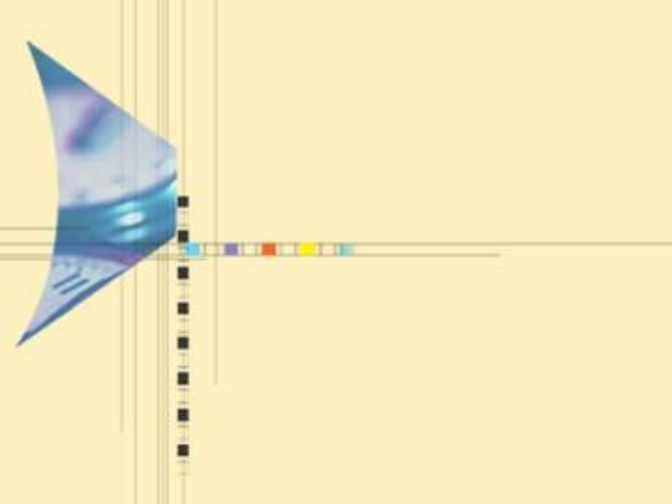




算法设计与分析

主讲：刘娟，武汉大学人工智能学院

2024-2025 第二学期，1-16周

周一(3-4，三区1-502)，周三（6-7，三区1-325）

2025/4/1

武汉大学计算机学院

武汉大学
Wuhan University





主要内容

5.1 引言

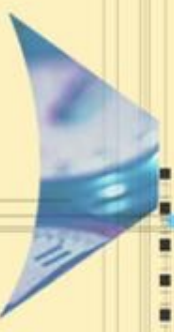
5.2 贪心算法简单例

5.3 贪心法设计思想

5.4 贪心法正确性证明

5.5 对贪心法得不到最优解情况的处理

5.6 贪心策略的典型应用



主要内容

5.6 贪心算法典型应用（选讲）

5.6.1 文件压缩问题

5.6.2 最优归并模式问题

5.6.3 带期限作业排序问题

5.6.4 蓄栏保留问题

5.6.5 货郎担问题

5.6.6 集合覆盖问题

5.6.1 文件压缩

问题描述:

假设有一个字符串文件，设文件中的字符集是 $C = \{c_1, c_2, \dots, c_n\}$ ，又设 $f(c_i)$, $1 \leq i \leq n$ ，是文件中字符 c_i 的频度，即文件中 c_i 出现的次数。

希望用一种方法对文件中每个字符进行0-1形式的编码，要求尽可能多地压缩文件，但源文件能够很容易地被重建。如何对文件中字符进行编码，使得压缩文件的大小最小。

定长编码和变长编码

定长编码：用定长比特数表示每个字符，称为字符的编码，文件的大小取决于文件中的字符数。由于有些字符的频度可能远大于另外一些字符的频度，将频率高的字符采用频率低的字符一样的长度进行编码使得存储资源占用大。

变长编码：从直观上，节省存储资源的方式是采用变长方编码方式，即那些频度大的字符将被赋予短的编码，而长的编码可以赋给那些频度小的字符。

变长编码的歧义性

- 由于字符的编码长度不同，可能出现一个字符的编码是另一个字符编码的前缀（即词头），这种码称为**前缀码**。前缀码的存在会造成在重建文件时的二义性。
 - 例如，如果我们把编码10和101赋予字符“a”和“b”就会存在二义性，不清楚10究竟是“a”的编码还是字符“b”编码的前缀。
- 变长编码的前缀约束：当编码在长度上变化时，规定一个字符的编码必须不能是另一个字符编码的前缀码。
- 如何满足前缀约束？=> 构造满二叉树形式的编码树

满二叉树形式的编码树

- 树的每个内部节点刚好有两个分支，分别标记为0和1；
- 树的叶节点对应文件中的不同字符。
- 从树的根节点到叶节点的每一条路径上的0和1序列对应一个字符的编码。
- 如何构造一棵具有最小化压缩文件大小的编码树？

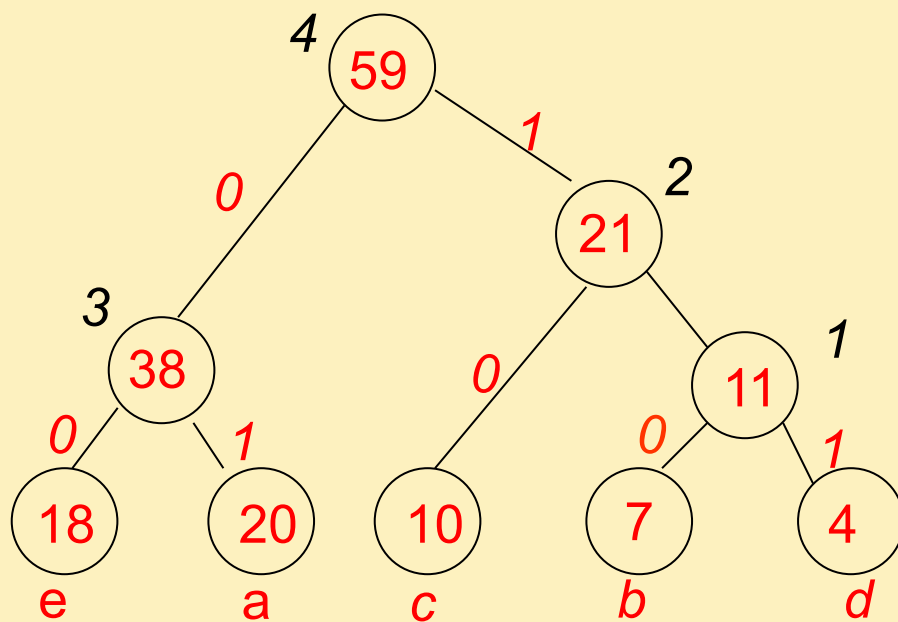
Huffman 树

- 由Huffman提出，算法构造的编码满足前缀约束，并且最小化压缩文件的大小。
- 算法重复下面过程，直到字符集C仅由一个字符组成：
 - 设 c_i 和 c_j 是C中两个具有最小频度的字符，建立一个新节点 c ，它的频度 c_i 和 c_j 的频度之和，
 - 使 c_i 和 c_j 成为 c 的子节点，令 $C = C - \{c_i, c_j\} \cup \{c\}$

例子

- 文件由字母a, b, c, d和e组成, 频度为:

$$f(a) = 20, f(b) = 7, f(c) = 10, f(d) = 4 \text{ and } f(e) = 18$$



- 设压缩前每个字符3个bit, 文件大小为3*(20+7+10+4+18)=177
- 压缩后文件大小为20*2+7*3+10*2+4*3+18*2=129
- 节省约27%

a, b, c, d和e编码: 01,110,10,111,00

算法5.9 HUFFMAN

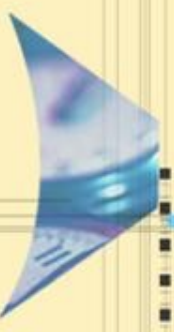
输入: n 个字符的集合 $C=\{c_1, c_2, \dots, c_n\}$ 和它们的频度 $\{f(c_1), f(c_2), \dots, f(c_n)\}$.

输出: C 的 Huffman 树 (V, T)

1. 根据频度将所有字符插入最小堆 H
2. $V \leftarrow C; T = \{\}$
3. for $j \leftarrow 1$ to $n-1$
4. $c \leftarrow \text{DELETEMIN}(H)$
5. $c' \leftarrow \text{DELETEMIN}(H)$
6. $f(v) \leftarrow f(c) + f(c')$ $\{v \text{ 是一个新节点}\}$
7. $\text{INSERT}(H, v)$
8. $V = V \cup \{v\}$ $\{\text{添加 } v \text{ 到 } V\}$
9. $T = T \cup \{(v, c), (v, c')\}$ $\{\text{使 } c \text{ 和 } c' \text{ 成为 } T \text{ 中 } v \text{ 的孩子}\}$
10. end for

HUFFMAN算法分析

- 把所有字符插入堆中需要时间 $\Theta(n)$ 。
- 从堆中删除两个元素和加一个新元素需要的时间是 $O(\log n)$ 。
- 因为这被重复 $n-1$ 次，for循环需要的所有时间是 $O(n \log n)$ ，
- 算法的时间复杂度是 $O(n \log n)$



Huffman算法正确性证明

命题1: Huffman算法得到一棵树

证明:

- 根据算法, 初始时, $|V|=n$, $|T|=0$ 。表示 n 棵树组成的森林;
- 算法每次迭代, 将两棵树合并为一棵树。也就是森林中树的数目减1;
- 算法一共重复 $n-1$ 次; 因此, 最后只剩下一棵树。

Huffman算法正确性证明

命题2: Huffman算法得到的是带权路径长度最小的树。

证明:

- 设通过Huffman算法得到的树为 T_1 , 由算法可知, 字符权越小, 在树中的深度越深。设节点 $a, b, f(a) < f(b)$, 在 T_1 中深度不同, 从根节点到两节点的路径长度分别为 $d_1, d_2, d_1 > d_2$ 。记 T_1 中除这两节点外其他节点带权路径总长为 X 。因此有:

$$T_1 = f(a) * d_1 + f(b) * d_2 + X$$

- 交换 T_1 中的 a, b 位置, 得到 T_2 , 显然有: $T_2 = f(a) * d_2 + f(b) * d_1 + X$
- $T_1 - T_2 = f(a) * d_1 + f(b) * d_2 - f(a) * d_2 - f(b) * d_1$

$$= f(a) * (d_1 - d_2) + f(b) * (d_2 - d_1) = (f(a) - f(b)) * (d_1 - d_2) < 0$$

因此, $T_1 < T_2$ 。即Huffman算法得到带权路径长度最小的树

5.6.2 最优归并模式问题

- 假设两个分别包含 n 个和 m 个记录的已分类文件可以在 $O(n+m)$ 时间内归并在一起而得到一个分类文件。当要把两个以上的已分类文件归并在一起时，可以通过成对地重复归并已分类的文件来完成。

例如，假定 X_1, X_2, X_3, X_4 是要归并的文件，则：

- 可以首先把 X_1 和 X_2 归并成文件 Y_1 ，然后将 Y_1 和 X_3 归并成 Y_2 ，最后将 Y_2 和 X_4 归并，从而得到想要的分类文件；
- 也可以先把 X_1 和 X_2 归并成 Y_1 ，然后把 X_3 和 X_4 归并成 Y_2 ，最后归并 Y_1 和 Y_2 而得到想要的分类文件。

5.6.2 最优归并模式

- 不同的归并方法，要进行的文件记录移动次数是不同的。
- 例如：假设三个文件X1,X2,X3分别含有30,20,10个记录。
 - 先合并X1,X2需要 $30+20=50$ 次记录移动操作后得到含50个记录的文件；然后将合并后的新文件和X3合并，需要 $50+10=60$ 次记录移动。因此总移动次数为 $50+60=110$ 。
 - 先合并X2,X3,需要 $20+10=30$ 记录移动得到大小为30的文件；然后和X1合并，需要 $30+30=60$ 次记录移动。因此总移动次数为 $30+60=90$ 。

问题描述：要求将n个已经分类的文件归并成一个分类好的单一文件，求最优的归并模式（移动总次数最少）。

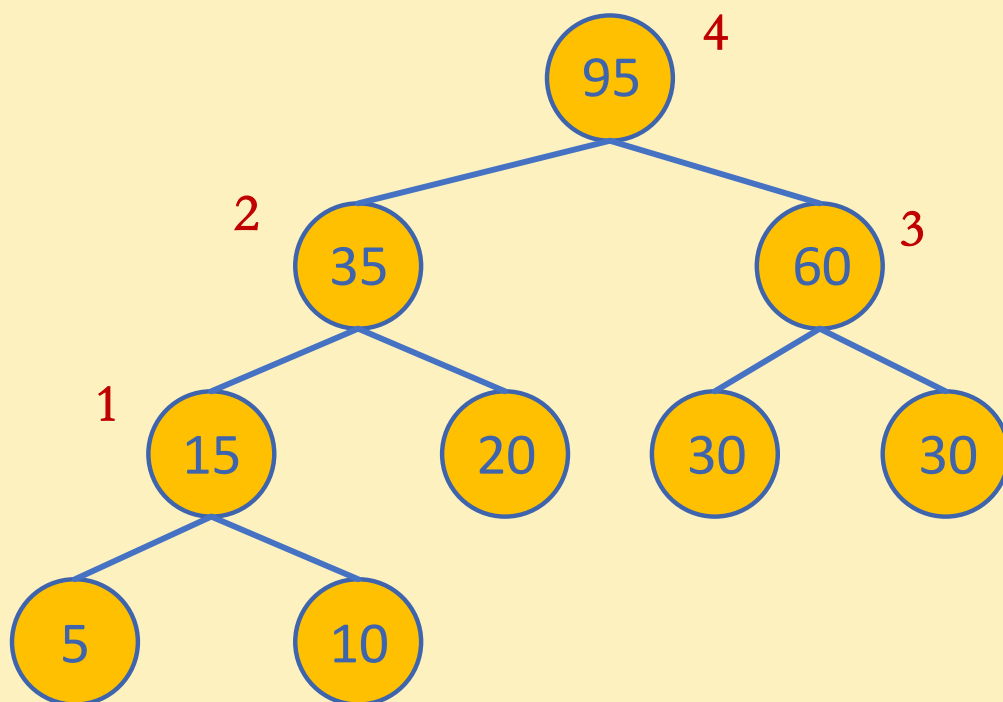
贪心策略

- 采用二元归并模式：每次选择包含记录最少的两个文件进行归并。
- 用二元归并树来表示二元归并模式。
 - 树的每个内部节点刚好有两个儿子，表示把它的两个儿子所表示的文件归并得到的文件；
 - 树的叶子结点代表要合并的文件；
 - 每个结点中的数字代表那个结点所表示的文件大小（记录数）。

Huffman树的构造类似！

实例

- 将5个长度分别为20, 30, 30, 10, 5的有序文件F1, F2, F3, F4, F5 两两合并成一个有序文件。



移动总次数=

$$3 * (5 + 10) + 2 * (20 + 30 + 30) = 45 + 160 = 205$$

二元归并思想（Huffman树构造）

- (1) 构造 n 棵二叉树的集合 $F = \{T_1, T_2, \dots, T_n\}$ ，其中每棵二叉树中均只含一个带权值的根结点，其左、右子树为空树。根结点代表要归并的文件，其权值为文件的大小；
- (2) 在 F 中选取其根结点的权值为最小的两棵二叉树，分别作为左、右子树构造一棵新的二叉树，并置这棵新的二叉树根结点的权值为其左、右子树根结点的权值之和；
- (3) 从 F 中删去这两棵树，同时加入刚生成的新二叉树；
- (4) 重复(2)和(3)两步，直至 F 中只含一棵树为止。

最优归并模式

- 树的带权路径长度：树中所有叶子结点的带权路径长度之和。
- 最优度量标准：带权路径长度最小
- Huffman树是最优归并模式

5.6.3带期限作业排序问题

带有期限的作业排序要解决的是操作系统中单机、无资源约束且每个作业可在等量的时间内完成的作业调度问题。

问题描述：

- 要在一台机器上处理 n 个作业，每个作业均可在单位时间内完成；
- 每个作业 i 都有一个期限值 d_i （整数）， $d_i > 0$ ；
- 当且仅当作业 i 在它规定的期限值前完成时可以获得 $p_i > 0$ 的效益；
- 求一个作业子集 J ， J 中的每个作业都能在各自的截止期限前完成，并且 J 中作业效益值的和最大。

实例

$n=4$, $(p_1, p_2, p_3, p_4) = (100, 10, 15, 20)$, $(d_1, d_2, d_3, d_4) = (2, 1, 2, 1)$ 。

问题的可行解、处理顺序、效益值为：

1	{1}	1	100
2	{2}	2	10
3	{3}	3	15
4	{4}	4	20
5	{1, 2}	2, 1	110
6	{1, 3}	1, 3或3, 1	115
7	{1, 4}	4, 1	120
8	{2, 3}	2, 3	25
9	{3, 4}	4, 3	35

这个问题的最优解为第7个。

所允许的处理顺序是：

在时间0开始处理作业4，而在时间2完成对作业1的处理。



贪心选择标准

- 由于目标是选中的作业的效益值的和最大。因此每次选择在满足可行性前提下让效益值增加最多的作业。
- 贪心准则：每次选择效益值最大的作业。



算法框架

算法 GREEDY-JOB(D, J, n)

//作业已按照效益值降序排列: $p_1 \geq p_2 \geq \dots \geq p_n$ 的次序输入, 它们的期限值 $D[i] \geq 1, 1 \leq i \leq n, n \geq 1$ 。J是在它们的截止期限前完成的作业集合。

1. $J \leftarrow \{1\}$
2. for $i \leftarrow 2$ to n
3. if $J \cup \{i\}$ 所有作业都能在截止期限前完成 then
4. $J \leftarrow J \cup \{i\}$
5. end if
6. end for
7. return J

实例

- $n=4$, $(p_1, p_2, p_3, p_4) = (100, 10, 15, 20)$, $(d_1, d_2, d_3, d_4) = (2, 1, 2, 1)$ 。
- 把作业按照它们的效益值作一个非增次序的排序。
- 作业根据效益值排序为: p_1 、 p_4 、 p_3 、 p_2 。作业对应截止期排序为: d_1 、 d_4 、 d_3 、 d_2 。
- 作业1加入J, 由于作业1的期限值 $d_1=2$, 故 $J=\{1\}$ 是一个可行解。
- 考虑作业4。由于作业4的期限值 $d_4=1$ 而作业1的期限值 $d_1=2$, 可以先完成作业4后再完成作业1, 故 $J=\{1, 4\}$ 是一个可行解。
- 再考虑作业3, 但是作业3的期限值 $d_3=2$, 计入J后就没有办法保证J中的每一个作业都在期限值内完成, 因此计入作业3后不能构成一个可行解。同理计入2后也不能构成一个可行解。
- 因此, 最后得到 $J=\{1, 4\}$, 也是问题的最优解。

GREEDY-JOB正确性证明

定理 算法GREEDY-JOB总能得到最优解。

证明：

- 假设最优解的作业集合为 I ，贪心解的作业集合为 J 。
- 如果 $I=J$ ，显然有 J 是最优解；
- 假定 $I \neq J$ ，容易看出 $I \not\subseteq J$ ，否则 I 不可能是最优解；同时，也有 $J \not\subseteq I$ ，否则 J 是最优解；
- 因此，至少存在两个作业 a 和 b ， $a \in J-I$ ， $b \in I-J$ 。设 a 是 $J-I$ 中的具有最高效益值的作业。由于 $b \in I-J$ ，根据贪心法的工作方式可知，一定有 $p_a \geq p_b$ 。
- 现在，设 S_J 和 S_I 分别是 J 和 I 中作业的可行调度表。设作业 $i \in I \cap J$ ；又设 i 在 S_J 中在 $[t, t+1]$ 时刻被调度，而在 S_I 中在 $[t', t'+1]$ 时刻被调度。

GREEDY-JOB正确性证明

- 如果 $t < t'$ ，则可以将 S_J 中 $[t', t'+1]$ 时刻所调度的作业（如果有的话）与 i 交换。如果 S_J 中 $[t', t'+1]$ 没有作业被调度，则将 i 移到 $[t', t'+1]$ 时刻调度。这样形成的调度表也是可行的。
- 如果 $t' < t$ ，则对 S_I 中作业做类似调换，得到可行调度表。
- 用这种方法，就能得到两个新的可行调度表 S_J' 和 S_I' 。使得 J 和 I 中共有的所有作业都在相同的时间被调度。
- 现考虑在新的调度表 S_J' 中的作业 a 和 S_I' 中的作业 b 。设 S_J' 中 $[t_a, t_a+1]$ 时刻，作业 a 被调度，在 S_I' 中 $[t_a, t_a+1]$ 时刻作业 b 被调度。在 S_I' 中的 $[t_a, t_a+1]$ 时刻将作业 b 替换成调度作业 a ，得到一个关于作业集合 $I' = I - \{b\} \cup \{a\}$ 可行的调度。由于 $p_a \geq p_b$ ，因此 I' 的效益值不小于 I 的效益值，因此 I' 也是最优作业集合，并且 I' 比 I 少一个与 J 不同的作业。
- 重复上述过程，将使 I 在不减少效益值的情况下转换成 J ，因此 J 也是最优解。

J的可行性判定

- 对于一个给定的作业集合J，为了判定J是否可行，蛮力方法是穷举J中作业的所有可能排列次序，对于任何一个次序，判定这些作业是否能在其期限前完成。假定J中有k个作业，则需要 $O(k*k!)$ 时间。
- 事实上，对于J的可行性，只需要检验J中作业的一个特殊排列来确定，即按期限的非降次序的排列。

定理 设J是k个作业的集合， $\sigma=i_1, i_2, \dots, i_k$ 是J中作业的一种排列，使得 $d_{i_1} \leq d_{i_2} \leq \dots \leq d_{i_k}$ 。J是一个可行解，当且仅当J中作业可以按 σ 的次序而又不违反任何一个期限的情况来处理。

算法5.10 JS(D,J,n,k)

//作业已按照效益值降序排列: $p_1 \geq p_2 \geq \dots \geq p_n$ 的次序输入, 它们的期限值 $D[i] \geq 1, 1 \leq i \leq n, n \geq 1$ 。J[i]是最优解中的第i个作业, $k \geq 1$ 是当前J中作业数, 终止时, $D[J[i]] \leq D[J[i+1]]$ 。

1. $J[0] \leftarrow 0; D[0] \leftarrow 0$ //引入虚构作业0,截止期为0。便于将作业插入位置1
2. $k \leftarrow 1; J[1] \leftarrow 1$ //计入作业1
3. **for** $i \leftarrow 2$ **to** n //按p的非增次序考虑作业i, 找i的位置并检查插入可行性
4. $r \leftarrow k$ //从J中最后一个作业开始依次往前检查找i的插入位置
5. **while** $D[J[r]] > D[i]$ **and** $D[J[r]] \neq r$ //找位置不可能后移的作业位置r
6. $r \leftarrow r - 1$
7. **end while**
8. **if** $D[J[r]] \leq D[i]$ **and** $D[i] > r$ **then** //如i可在位置r后插, 则可行
9. **for** $i \leftarrow k$ **to** $r + 1$ //从位置k到位置r+1的作业后移1个得i的位置
10. $J[i + 1] \leftarrow J[i]$
11. **end for**
12. $J[r + 1] \leftarrow i; k \leftarrow k + 1$ //插入作业i到r+1的位置
13. **end if**
14. **end for**



JS算法的分析

- 最外层3~14行for循环执行 $n-1$ 次
- 内层5~7行while循环至多 k 次
- if条件内 9~11行for循环，条件为真时执行 $k-r$ 次
- 算法的时间复杂度为 $O(ns)=O(n^2)$, s 为最后求得的 J 中作业数。
- 算法需要 $O(n)$ 空间保存解 J



JS算法的改进

- 基本思路：如果 J 是作业的可行子集，尽可能推迟对其中每个作业的处理。
- 具体操作：若还没给作业 i 分配处理时间，则分配给它时间片 $[\alpha-1, \alpha]$ ，其中 α 尽可能大且时间片 $[\alpha-1, \alpha]$ 是空的。这样，在作业一个一个装配到 J 中时，不必要为接纳新作业而取移动 J 中那些已分配了时间片的作业。如果考虑的作业不存在上述定义的 α ，则这个作业不能计入 J （正确性自己证明）。

实例

设 $n=5$; $p[1..5]=[20,15,10,5,1]$; $D[1..5]=[2,2,1,3,3]$, 得

J	已分配时间片	正被考虑作业	动作
Φ	无	1	分配 [1,2]
{1}	[1,2]	2	分配 [0,1]
{1,2}	[0,1],[1,2]	3	不适合, 舍弃
{1,2}	[0,1],[1,2]	4	分配[2,3]
{1,2,4}	[0,1],[1,2],[2,3]	5	不适合, 舍弃

调度规则的实现

基本思路:

- 由于每个作业花费一个单位时间, 因此只需要考虑一些时间片 $[i-1, i]$, $1 \leq i \leq b$, 其中 $b = \min\{n, \max\{d_i\}\}$, 为简单起见, 不妨用 i 表示时间片 $[i-1, i]$ 。这样, n 个作业的期限只能是 $\{1, 2, \dots, b\}$ 中的元素。
- 把 b 个期限值分成一些集合。对于任一期限值 i , 设 $F(i)$ 是使得 $F(i) \leq i$ 的最大整数且是空的时间片。同样, 为避免极端情况, 引入虚构期限值 0 和时间片 $[-1, 0]$ 。
- 当且仅当 $F(i) = F(j)$ 时, 期限 i 和 j 在同一个集合中, 即如果所要处理的作业期限值为 i 或者 j , 当前可分配的最接近的时间片是 $F(i)$ ($=F(j)$)。显然, 如果 $i < j$, 则 $i, i+1, \dots, j$ 都在一个集合中。

调度规则的实现

基本思路（续）：

- 也就是，构造一些以期限值为元素的集合，同一个集合中的元素都有个当前共同可用的最接近的时间片。
- 利用不相交集表示法，采用树来表示集合。
- 如果要调度期限值为 d 的作业，寻找包含期限值 $\min(n, d)$ 的树的根，如果根为 j ，且 $F(j) \neq 0$ ，则 $F(j)$ 是最接近时间片。使用了这个时间片后，其根为 j 的集合应该与包含期限值 $F(j)-1$ 的集合合并。

算法5.11 FJS(D, n, J)

//作业已按照效益值降序排列: $p_1 \geq p_2 \geq \dots \geq p_n$ 的次序输入, 它们的期限值 $D[i] \geq 1, 1 \leq i \leq n, n \geq 1$ 。

```
1.  $b \leftarrow \min\{n, \max\{D[i]\}\}$ 
2. for  $i \leftarrow 0$  to  $b$  //
3.   MAKESET( $i$ ) //每个时间片都是一棵树
4.    $F[i] \leftarrow i$  //  $F[i] \neq 0$ , 则是  $i$  所在树最接近时间片
5. end for
6.  $k \leftarrow 0$  //初始时  $J$  中没有作业
7. for  $i \leftarrow 1$  to  $n$  //贪心规则依次考虑每个作业
8.    $j \leftarrow \text{FIND}(\min(n, D[i]))$  //从作业  $i$  截止期所在的树  $j$ 
8.   if  $F[j] \neq 0$  then
10.     $k \leftarrow k + 1; J[k] \leftarrow i$  //选择作业  $i$ 
11.     $m \leftarrow \text{FIND}(F[j] - 1)$ 
12.    UNION( $m, j$ )
13.     $F[j] \leftarrow F[m]$ 
14.   end if
15. end for
```

时间复杂度为 $\Theta(n)$



5.6.4 蓄栏保留问题

问题描述：

农场有 n 头牛，每头牛会有一个特定的时间区间 $[b, e]$ 在蓄栏里挤牛奶，一个蓄栏里任何时刻只能有一头牛挤奶。

现在农场主希望用最少的蓄栏能够满足上述要求，并给出每头牛被安排的方案。多种可行方案时输出一种即可。

问题建模基本思路

设牛的编号为 $1 \sim n$ ，安排每头牛的挤奶时间相当于安排一个活动，那么每个蓄栏相当与一个活动场所。因此可用活动选择算法为每个蓄栏安排最多的兼容活动子集。

可以利用活动选择算法求该问题

算法思想

- (1) 采用与求解活动安排问题类似的贪心思路，将所有奶牛这样排序：如果结束时间相同按开始时间递增排序，否则按结束时间递增排序。
- (2) 利用活动选择算法求出一个最大兼容奶牛挤奶活动子集，将它们安排在一个蓄栏中挤奶（蓄栏编号为1）；
- (3) 如果奶牛没有安排完，再对剩余的奶牛再求下一个最大兼容奶牛挤奶活动子集，将它们安排在另一个蓄栏中（蓄栏编号为2）
- (4) 以此类推，直到安排完所有奶牛。
- (5) 最后得到的最大兼容活动子集的个数就是需要的最少蓄栏个数。

实例

i	1	2	3	4	5	6	7
开始时间	1	2	5	8	4	12	11
结束时间	4	5	7	9	10	13	15



1	3	4	6	2	7	5
1	5	8	12	2	11	4
4	7	9	13	5	15	10

最大兼容活动子集个数为3

5.6.5 货郎担问题

- 货郎担问题也叫旅行商问题，即TSP问题（Traveling Salesman Problem），是数学领域中著名问题之一。
- **问题描述：**某售货员要到若干个城市销售货物，已知各城市之间的距离，要求售货员选择出发的城市及旅行路线，使每个城市经过一次，最后回到原出发城市，而总路程最短。
- 相当于求距离最短的哈密尔顿回路，需求从各城市出发的最短回路，再对 n 条回路求极小值。
- 回路最短——贪心选择距离短的路线

5.6.5 货郎担问题

例：假设5个城市的费用矩阵为：

	1	2	3	4	5
1	∞	3	3	2	6
2	3	∞	7	3	2
3	3	7	∞	2	5
4	2	3	2	∞	3
5	6	2	5	3	∞

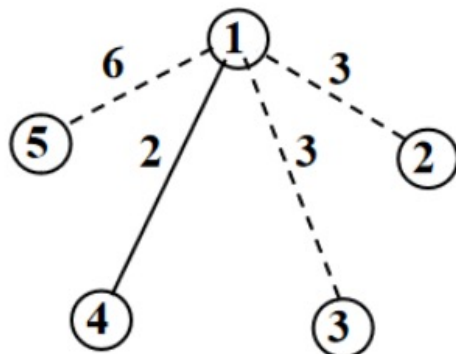
按最近邻点策略

- 从城市1出发：1->4->3->5->2->1，总距离14
- 从城市2出发：2->5->4->1->3->2，总距离17
- 其它城市同理，再在5条回路中选择最小距离。

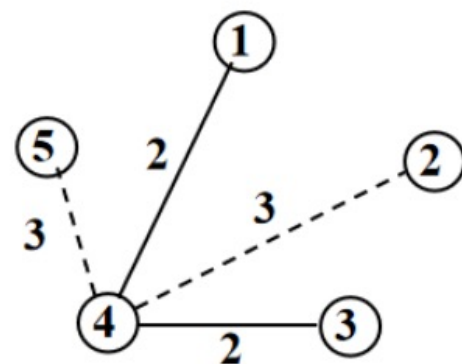
最近邻点贪心策略

$$C = \begin{pmatrix} \infty & 3 & 3 & 2 & 6 \\ 3 & \infty & 7 & 3 & 2 \\ 3 & 7 & \infty & 2 & 5 \\ 2 & 3 & 2 & \infty & 3 \\ 6 & 2 & 5 & 3 & \infty \end{pmatrix}$$

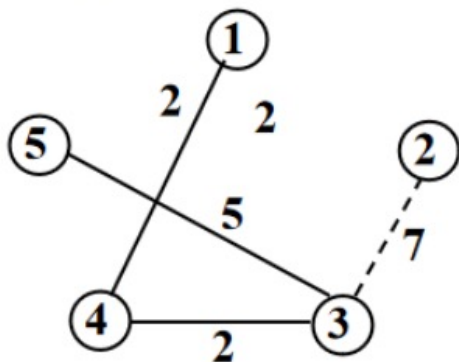
(a) 5城市的代价矩阵



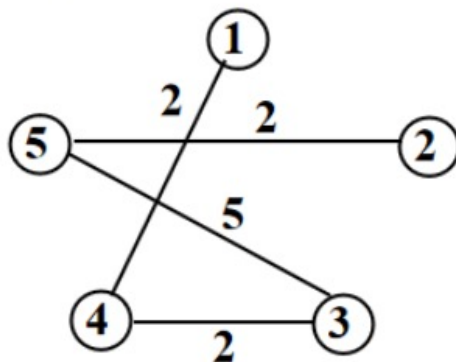
(b) 城市1→城市4



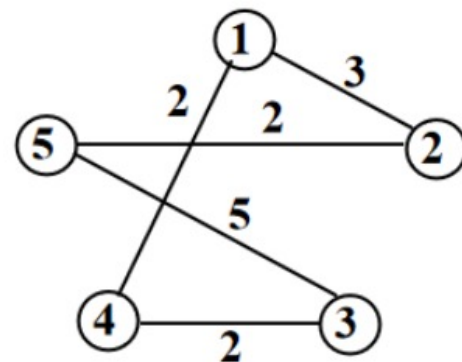
(c) 城市4→城市3



(d) 城市3→城市5



(e) 城市5→城市2



(f) 城市2→城市1

最近邻点贪心策略求解TSP问题的过程，求出的最短路径为1-4-3-5-2-1，长度为14

时间复杂度

- 从一个城市出发的最短回路计算为 $O(n^2)$,
- 共 n 个城市
- 故时间复杂度为 $O(n^3)$

远小于穷举法的 $n!$

最优解分析

- 最优解分析可得:

从城市1出发最优路线:1->2->5->4->3->1, 距离13

- 货郎担问题不具有贪心选择性生质和最优子结构性。只能得到近似解, 不能得到最优解。

5.6.6 集合覆盖问题

- **集合覆盖问题** (Set covering problem, SCP) 是组合数学、计算机科学和计算复杂性理论中的一个经典问题。集合覆盖的判定问题是卡普的二十一个NP-完全问题之一。
- **问题描述**: 一个集合U以及U内元素构成的若干个子集的集合S, 目标是找到S的一个最小子集, 该子集包含了U中的所有元素。
- 例如, $U=\{1,2,3,4,5\}$, $S=\{\{1,2\},\{3,4\},\{2,4,5\},\{4,5\}\}$, 找到S子集能满足条件的可以有 $O=\{\{1,2\},\{3,4\},\{4,5\}\}$ 或是 $O=\{\{1,2\},\{3,4\},\{2,4,5\}\}$

5.6.6 集合覆盖问题

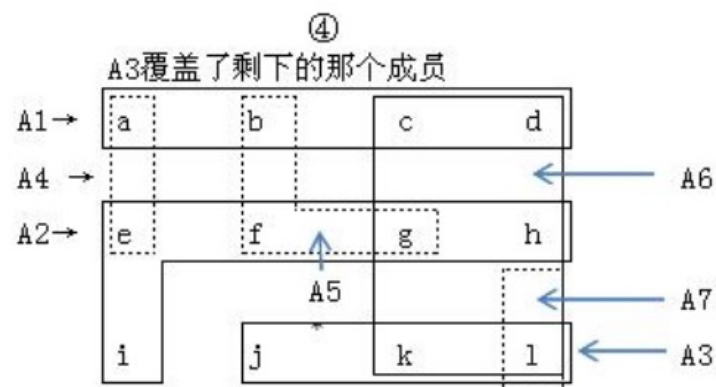
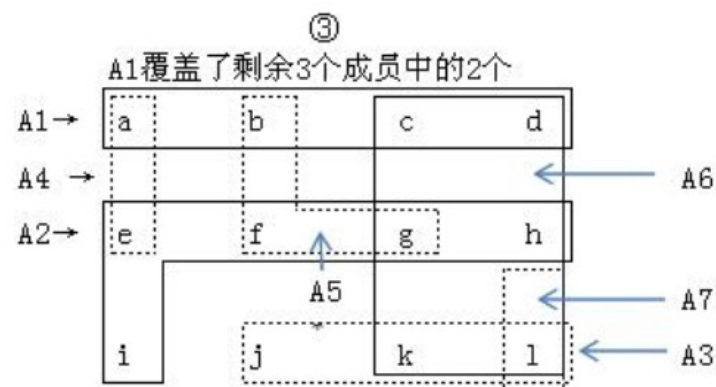
- **方法1-蛮力穷举搜索**: 对需要的最少集合个数变量 k 从1到 m 进行遍历 (m 为 S 中子集的个数)。遍历 $C(m,k)$ 种集合组合情况。对每种情况求集合并集, 观察是否满足全元素覆盖; 找到第一个满足条件的情况时结束搜索, 输出 k 。
- 特点是可求出确切解 (exact solution), 但是当 k 值较大时组合空间过大且涉及频繁的集合求并效率极低。
- **方法2-贪心策略**: 每次迭代添加包含未覆盖元素最多的集合, 直到满足全覆盖条件。
- 特点是速度快但只是近似解而非确切解。

例子

- 设有12种技能的集合 $S=\{a,b,c,d,e,f,g,h,i,j,k,l\}$ 。有7名待选选手的集合 $P=\{A1,...A7\}$ 。P中选手拥有的技能集合分别为：
- $A1=\{a,b,c,d\}$ ， $A2=\{e,f,g,h,i\}$ ， $A3=\{j,k,l\}$ ， $A4=\{a,e\}$ ， $A5=\{b,f,g\}$ ， $A6=\{c,d,g,h,k,l\}$ ， $A7=\{l\}$ 。
- 求能覆盖所有技能的最小选手集合。
- 最佳覆盖集应该是 $C=\{A1,A2,A3\}$ 。
- 贪心算法给出的算法选择的集合是 $C=\{A6,A2,A1,A3\}$ 。

例子

- $A1=\{a,b,c,d\}$, $A2=\{e,f,g,h,i\}$, $A3=\{j,k,l\}$, $A4=\{a,e\}$, $A5=\{b,f,g\}$, $A6=\{c,d,g,h,k,l\}$, $A7=\{l\}$ 。



贪心法小结

1. 适用于组合优化问题. 求解过程是多步判断. 判断的依据是局部最优策略, 使目标值达到最大(或最小), 与前面的子问题计算结果无关.
2. 问题求解表示成多步判断; 整个判断序列对应问题的解, 子序列对应子问题的解.
3. 判断的依据-贪心选择: 短视的局部最优策略. 策略的选择是算法能否得到最优解的关键, 需要证明.
4. 正确性证明方法: 数学归纳法、交换论证.
 - 使用数学归纳法主要通过对算法步数或者问题规模进行归纳.
 - 如果要证明贪心策略是错误的, 只需举出反例.
5. 自顶向下求解, 通过选择将问题归约为小的子问题.
6. 如果贪心法得不到最优解, 可以对问题的输入进行分析或者估计算法的近似比.
7. 算法时间取决于判断步数和每步的工作量. 如果对原始数据排序之后, 贪心法往往是一轮处理, 时间复杂度和空间复杂度低.